COMPETITIVE AI SOLVER FOR CSUDOKU

AI Project Report

Vadim Kotkin and Tair Baurzhan

Université Paris-Saclay | Selected Advanced Coursework | Java and Game AI

English translation of the original French report

TABLE OF CONTENTS

1. IMPLEMENTATION OF BASELINE PLAYERS

1.1 Random move without validation

This simple strategy randomly selects a row, column, and value (`RandomMoveNoValidationStrategy`, lines 38, 39, and 65). If the selected cell is occupied, the strategy continues generating random moves. To prevent an infinite loop, the number of attempts is capped at the cube of the board size. If no move is found within that limit, the strategy falls back to a systematic search.

1.2 Random move with validation

This strategy follows the same process as the unvalidated random strategy, but it calls `Player.isValidMove` to verify every generated move. The loop is again capped at the cube of the board size, and the collection of empty cells is shuffled before exploration.

1.3 First valid move

The strategy starts at the first row and first column, scans the entire row, and then continues row by row until it finds a valid move. It returns `null` if no valid move exists.

2. MINIMAX

2.1 Minimax strategy

The initial search depth is set to three. In `selectMove`, the best value is initialized to negative infinity, and an `EvaluatedSimulatedBoard` is constructed from the current board. For every valid move, the strategy applies the move, calls Minimax recursively for the opposing player at a lower depth, and then restores the previous board state. The move with the best returned evaluation is selected.

2.2 Recursive Minimax method

The recursive `minimax` method receives the current simulated board, remaining search depth, a flag indicating whether the active player is maximizing, and a reference to the AI player.

- Increment the visited-node counter.
- Stop when the depth reaches zero or the board is complete.
- Generate all valid moves.
- Initialize the best value to negative or positive infinity according to the active role.
- Evaluate each move recursively while decreasing the depth and alternating the maximizing role.
- Undo each move after evaluation; the previous cell value is retained for backtracking.
- Return the maximum score for the AI player or the minimum score for the opponent.

2.3 Evaluated simulated board

The `EvaluatedSimulatedBoard` class extends `CSudokuBoard` and adds:

- Tracking of empty cells by row and column through `zerosInRows` and `zerosInColumns`.
- A heuristic evaluation value, `eval`.
- The most recent move, stored as `lastMove`.
- A copy constructor that duplicates board state, constraints, and empty-cell counters.

2.4 Board evaluation

The `setValue` method updates the evaluation incrementally. The numeric value of the move provides the base score. A bonus equal to the square of the board size is added when the move completes a row, column, or subgrid. The score is added for the active player and subtracted for the opponent. Empty-cell counters and validity checks are updated at the same time, with diagnostic logging for potential errors.

2.5 Performance measurements

- Maximum search depth fixed at three to balance decision quality and execution time.
- Visited-node counting for every search.
- Execution-time measurement for move selection.

3. ALPHA-BETA PRUNING

3.1 Principle

Alpha-Beta pruning improves Minimax by eliminating branches that cannot influence the final decision. The search depth remains fixed at three. Alpha and beta represent the best guaranteed values for the maximizing and minimizing players. An `AlphaBetaPruningObserver` records visited nodes, alpha cutoffs, and beta cutoffs.

3.2 Move selection

The method begins like Minimax, then initializes alpha to negative infinity and beta to positive infinity. Each valid move is applied, evaluated recursively, and undone. The best value is updated, and alpha becomes the maximum of its current value and the new evaluation.

3.3 Recursive Alpha-Beta search

- For a MAX node, evaluate every valid move, undo it, and retain the highest value.
- For a MIN node, evaluate every valid move, undo it, and retain the lowest value.
- Apply a beta cutoff for MAX or an alpha cutoff for MIN whenever beta is less than or equal to alpha.

4. MONTE CARLO TREE SEARCH

4.1 Overview

Monte Carlo Tree Search (MCTS) is suited to games with a large branching factor such as CSudoku. Instead of exhaustively exploring the tree like Minimax or Alpha-Beta, it uses randomized simulations to estimate move quality and incrementally concentrates the search on promising regions.

4.2 Node structure

- `state`: an `EvaluatedSimulatedBoard` representing the game state.
- `parent`: reference to the parent node; null for the root.
- `children`: child MCTS nodes.
- `move`: the move connecting the parent state to the node.
- `visitCount`: number of search paths that included the node.
- `totalScore`: cumulative score from simulations passing through the node.
- `unTriedMoves`: valid moves that have not yet been expanded.

4.3 Constants and UCB1

- The UCB1 exploration constant is set to sqrt(2).
- The simulation limit is set to 100,000.

4.4 Move-selection cycle

For the configured number of simulations, `selectMove` executes four phases: selection, expansion, simulation, and backpropagation. The final move is chosen from the root's children by selecting the child with the highest visit count.

4.5 Selection

Starting from the root, the algorithm repeatedly selects the best child according to the UCB1 formula.

4.6 Expansion

If the selected node is non-terminal and not fully expanded, one move is chosen randomly from `unTriedMoves`. A new child is created and the move is removed from the untried list.

4.7 Simulation

A random game is played from the expanded node until a terminal state is reached. The final result is computed as the difference between the players' scores.

4.8 Backpropagation

The simulation result is propagated from the expanded node back to the root, updating the statistics of every node on the path.

5. TESTS

Before optimization, the complete test suite produced the following aggregate result:

Tests	Failures	Errors	Skipped	Total time
63	0	0	0	1 min 2 s

Test suite	Tests	Time
CSudokuBoardTest	9	0.051 s
HumanPlayerTest	4	1.452 s
EvaluatedSimulatedBoardTest	16	0.179 s
MinimaxMoveStrategyTest	4	53.178 s
AIPlayerTest	9	0.216 s
RandomMoveNoValidationStrategyTest	4	0.104 s
AutomatePlayerTest	4	0.014 s
FirstValidMoveStrategyTest	2	0.065 s
RandomMoveStrategyTest	5	0.051 s

Representative pre-optimization Minimax runs

Run	Initial moves	Time	Best move	Score
1	729	26,740 ms	(0, 0, 9)	9
2	729	26,416 ms	(0, 0, 9)	9

The original report notes that these tests were executed on a relatively low-powered personal computer.

6. ADDITIONAL TESTS

Additional Alpha-Beta and MCTS tests increased the suite to 71 tests with no failures, errors, or skipped tests.

Metric	Alpha-Beta run 1	Alpha-Beta run 2
Initial moves	729	729
Search depth	3	3
Visited nodes	2,526,578	2,526,578
Alpha cutoffs	720	720
Beta cutoffs	6,979	6,979
Execution time	82,743 ms	78,492 ms
Best move and score	(0, 0, 9), score 9	(0, 0, 9), score 9

The four Alpha-Beta tests completed in 161.246 seconds. Four MCTS strategy tests completed in 0.014 seconds.

7. PERFORMANCE AND COMPARISONS

7.1 Alpha-Beta and Minimax against a human player

Both strategies were tested in parallel on a 9 x 9 board. The human player moved first by placing 9 at coordinates (1, 1). Alpha-Beta was substantially faster because it explored far fewer nodes.

Metric	Alpha-Beta	Minimax
Visited nodes	12,662	470,937
Alpha cutoffs	8,513	Not applicable
Beta cutoffs	691	Not applicable
Best move	(0, 3, 9)	(0, 3, 9)
Evaluated score	9	9
Execution time	2,082 ms	36,324 ms

For the first response move, Alpha-Beta explored approximately 37 times fewer nodes and completed the search approximately 18 times faster. On the next human move, at (8, 7), the time difference was about 15 times and the node-count difference about 24 times. On the third move, the time difference fell to approximately 13 times as the remaining search space became smaller.

7.2 Alpha-Beta versus Minimax

In the reported simulations, the first player consistently won by a score of 98 to 90. Alpha-Beta produced the same game result while requiring substantially less time.

7.3 MCTS against a human player

MCTS was slightly faster in the reported comparison, requiring approximately 1,500 ms compared with about 2,000 ms for Alpha-Beta.

7.4 MCTS versus Alpha-Beta and the exploration constant

- With the UCB1 exploration constant set to sqrt(2), MCTS explores a wider range of moves and may begin with values such as 5 and then 3, whereas Alpha-Beta consistently prefers the highest-valued moves.
- When games stop before completion, reducing the exploration constant - for example to sqrt(2)/2 or 0.1 - generally favors Alpha-Beta, although the score difference narrows.
- Increasing the exploration constant reverses the tendency by encouraging broader MCTS exploration.

8. IMPLEMENTED OPTIMIZATIONS

8.1 Alpha-Beta

- Heuristic move sorting in `sortValidMoves` before exploration.
- Board reuse through `board.setValue` and `board.unSet` instead of allocating a new `EvaluatedSimulatedBoard` at every iteration.
- Use of `Collections.sort` for move ordering.

8.2 Minimax

- Parallelized search computations.
- A transposition table for caching previously evaluated states.
- Iterative Deepening Depth-First Search (IDDFS) with progressively increasing depth.
- Heuristic move ordering.

9. POST-OPTIMIZATION TESTS

Strategy	Execution time	Visited nodes	Reported improvement
Minimax	2,001 ms	546,816	Approximately 18x faster; node count increased
Alpha-Beta	1,079 ms	7,453	Approximately 2x faster; about 1.5x fewer nodes

Conclusion

The project demonstrates how algorithm choice and implementation details jointly determine game-AI performance. Alpha-Beta pruning dramatically reduced the effective search space compared with plain Minimax. MCTS offered competitive response times and different strategic behavior controlled by the UCB1 exploration constant. The final optimizations - particularly parallel execution, state caching, iterative deepening, move ordering, and reversible in-place board updates - produced substantial runtime improvements while preserving correctness across the full automated test suite.